

Multi Fractal Workflow: Transformation Information Geometry for PDDL 3.1 $\rightarrow$ POWL v2 Compilation

Complete Notation and Implementation Consequences
(Formalized in Lean 4, `ProcInt.MFW`, v26.7.13)

Sean Chatman

July 14, 2026

Abstract

Multi Fractal Workflow (MFW) studies the multiscale distribution of PDDL 3.1 lawful behavioral measures over the hierarchical behavior classes induced by POWL v2 transformation. The central formal object is a map $\tau : (\mathcal{P}(\Pi), d_{\mathcal{P}}, \nu) \rightarrow (\mathcal{W}, d_{\mathcal{W}}, \tau_*\nu)$ and the central open target is the *crown theorem* $\tau(b_1) = \tau(b_2) \iff b_1 \equiv_K b_2$, characterizing the kernel of the transformation as Mazurkiewicz trace equivalence. This paper transcribes the complete notation of the Lean 4 formalization (`procint/ProcInt/MFW`, 16 modules, compiled under Lean 4.31.0 with zero `sorry`) into standard mathematical form, records the proof standing of every object (PROVEN / DEFINITION / CONJECTURAL / STRUCTURALLY BLOCKED), and derives the consequences of each mathematical structure for the `ggen` implementation of the PDDL 3.1 $\rightarrow$ POWL v2 compiler. The paper states only what the formalization states: the crown biconditional and the entropy collapse are conjectural targets, not results.

Contents

1	Foundations: Planning Theory and Behavioral Phase Space	1
1.1	The planning theory Π	1
1.2	Behavior traces and lawfulness	2
1.3	The workflow space $\mathcal{W}$ (POWL v2)	2
1.4	The transformation, fibers, and induced equivalence	3
1.5	Hierarchy, pushforward, and the seven measures	3
2	Concurrency: Independence, Traces, Serialization Entropy	4
2.1	Independence	4
2.2	Mazurkiewicz traces	4
2.3	Causal partial orders and serialization entropy	4
2.4	Width, concurrency complex, abstraction ladder	5
3	The Kernel: Center of MFW	5
3.1	The four equivalence layers	6
3.2	The crown theorem	6

3.3	Consequences conditional on the crown	7
3.4	Kernel generators	7
4	Observability and the Information Horizon	8
5	Fiber Entropy (Layer 6)	8
6	Intrinsic Dimension Loss (Layer 7)	9
7	Observable Basis (Layer 8)	10
8	Spectrum Bundle and Cross-Spectrum Tension (Layer 9)	11
9	Workflow Geometry via Conditional Measures	13
10	Specification and Governance Modules	13
10.1	Manufacture (§§1–6, 15)	13
10.2	Falsification (§38)	13
10.3	Explore/Exploit (§§7–12, 35)	14
10.4	Search portfolio (QLens.lean §§31–35)	14
10.5	Compiler pipeline (§§36–37)	14
10.6	Central theorem ledger (§§39–40)	14
11	Verified Test Surface	14
12	How the Mathematics Shapes the Implementation	15
12.1	The kernel decides what the compiler may merge	15
12.2	Independence extraction is the load-bearing analysis	15
12.3	Fibers are disjoint: geometry and caching consequences	16
12.4	Entropy accounting as compiler telemetry	16
12.5	Observability gates what the generated system can report	16
12.6	The observable basis is the feature schema	16
12.7	Spectrum tension as the optimizer’s objective	17
12.8	Governance loop around the compiler	17
12.9	What is not yet load-bearing	17
13	Notation Index	17

1 Foundations: Planning Theory and Behavioral Phase Space

Source module: TransformBasic.lean (v2, post-audit 2026-07-14).

1.1 The planning theory Π

Definition 1.1 (PlanningTheory). [DEFINITION] A *PDDL 3.1 planning theory* is a tuple

$$\Pi = (S, A, \delta, s_0, G, \Theta, \models_T, N, \models_N, \Gamma, \models_\Gamma)$$

where

- S is the state space, A the action space;
- $\delta : S \times A \rightarrow S_\perp$ is the partial transition function (`Option State` in Lean; $\perp$ = inapplicable);
- $s_0 \in S$ is the initial state and $G \subseteq S$ the goal predicate;
- Θ is a type of temporal constraints with satisfaction relation $\models_T \subseteq \Theta \times \mathbb{R}^*$ over timestamp lists;
- N is a type of numeric fluents with effect map $A \times N \rightarrow \mathbb{R}$ and precondition satisfaction $\models_N \subseteq A \times (N \rightarrow \mathbb{R})$;
- Γ is a type of trajectory constraints (always, sometime, within, at-most-once) with satisfaction $\models_\Gamma \subseteq \Gamma \times S^*$ over state traces.

The audit rule recorded in the source: *a constraint type without a satisfaction law does not constrain the phase space* — every component of Π must carry semantics or be absent.

Definition 1.2 (Durative action expansion). [DEFINITION] A durative action over labels α expands into start/end temporal events

$$a = \langle a_+, a_-, I_a, \text{Inv}_a, \text{Eff}_a \rangle, \quad 0 \leq \underline{d}_a \leq \bar{d}_a,$$

with duration bounds $[\underline{d}_a, \bar{d}_a]$ (`DurativeActionExpansion`).

1.2 Behavior traces and lawfulness

The audit split the former monolithic behavior type into raw data, a predicate, and a proof-carrying subtype — the defect being repaired was that a “lawful” behavior with `achievesGoal := False` used to be an admitted inhabitant.

Definition 1.3 (BehaviorTrace). [DEFINITION] A *behavior trace* of Π is $b = (\mathbf{e}, \mathbf{t})$ with events $\mathbf{e} \in A^*$, timestamps $\mathbf{t} \in \mathbb{R}^*$, and $|\mathbf{e}| = |\mathbf{t}|$. Its induced *state trace* is the monadic fold

$$\text{stateTrace}(b) = \text{foldlM}(\delta, [s_0], \mathbf{e}) \in (S^*)_\perp,$$

which is $\perp$ if any action is inapplicable.

Definition 1.4 (IsLawful). [DEFINITION] b is *lawful* under Π , written $\text{IsLawful}_\Pi(b)$, iff there exists a state trace $\sigma \in S^*$ such that

- (i) $\text{stateTrace}(b) = \sigma$ and $|\sigma| = |\mathbf{e}| + 1$;
- (ii) the final state of σ satisfies G ;
- (iii) $\forall \theta \in \Theta$ of Π : $\theta \models_T \mathbf{t}$;
- (iv) $\forall \gamma \in \Gamma$ of Π : $\gamma \models_\Gamma \sigma$.

Definition 1.5 (LawfulBehavior and behavioral phase space). [DEFINITION] A *lawful behavior* is the dependent pair $\{b : \text{BehaviorTrace } \Pi // \text{IsLawful}_\Pi(b)\}$: raw trace plus lawfulness *witness*. The *behavioral phase space* is

$$\mathcal{P}(\Pi) = \text{LawfulBehavior}(\Pi).$$

Every inhabitant carries its proof; the proof is a witness, not a payload.

1.3 The workflow space $\mathcal{W}$ (POWL v2)

The audit replaced the former codomain $\mathbb{N}^3$ (which carried no partial order, choice graph, or hierarchy) with the actual POWL model.

Definition 1.6 (Powl model; Kourani & van Zelst, BPM 2023, Defs. 1–2). [DEFINITION] Over an activity label type α , a POWL model is inductively

$$\text{Powl}(\alpha) ::= \text{atom}(a) \mid \text{silent} \mid \text{xor}(\vec{c}) \mid \text{loop}(c_{\text{do}}, c_{\text{redo}}) \mid \text{po}(\vec{c}, \prec)$$

where $\vec{c}$ is a list of submodels and $\prec \subseteq \mathbb{N} \times \mathbb{N}$ a precedence on child indices. *Well-formedness* WF is hereditary and requires $|\vec{c}| \geq 2$ for xor, and irreflexivity plus transitivity of $\prec$ below the child count for po.

Definition 1.7 (Choice graph, POWL v2 object, workflow space). [DEFINITION] A *choice graph* is $(\vec{v}, E, |\vec{v}| \geq 2)$ with vertices $\vec{v} \in \text{Powl}(\alpha)^*$ and edge relation $E \subseteq \text{Fin}|\vec{v}| \times \text{Fin}|\vec{v}|$, modeling POWL v2’s non-block-structured decisions. A *POWL v2 semantic object* is

$$w = (m, \text{WF}(m), \text{depth} \in \mathbb{N}, \partial w \subseteq \mathbb{N} \text{ finite}, \vec{G}_{\text{choice}}),$$

carrying the model tree, well-formedness proof, hierarchical depth, semantic interface boundary variables ∂w , and optional choice graphs. The *workflow space* is $\mathcal{W} = \text{POWLv2Object}(\alpha)$.

1.4 The transformation, fibers, and induced equivalence

Definition 1.8 (WorkflowTransformation). [DEFINITION] The transformation is a bare forward map

$$\tau : \mathcal{P}(\Pi) \rightarrow \mathcal{W}.$$

Deliberately, *no* equivariance law is stored on the structure: the audit removed a vacuous `respects_equivalence` field, and the kernel characterization is a theorem target (§3), not an assumption.

Definition 1.9 (Fiber). [DEFINITION] $F_w = \tau^{-1}(w) = \{b \in \mathcal{P}(\Pi) : \tau(b) = w\}$.

Theorem 1.10 (fiber_partition). [PROVEN] $\forall b \in \mathcal{P}(\Pi) : b \in F_{\tau(b)}$.

Theorem 1.11 (fiber_disjoint). [PROVEN] $w_1 \neq w_2 \implies F_{w_1} \cap F_{w_2} = \emptyset$.

Remark 1.12 (The Jaccard falsifier). `fiber_disjoint` killed a proposed geometry before implementation: for distinct classes the Jaccard similarity is $J(F_{w_1}, F_{w_2}) = 0$, so the candidate metric $d_{\mathcal{W}}(w_1, w_2) = 1 - J$ collapses to the discrete metric. The repair is optimal transport over conditional measures (§9).

Definition 1.13 (Transformation equivalence). [DEFINITION] $b_1 \sim_{\tau} b_2 \iff \tau(b_1) = \tau(b_2)$. Reflexivity, symmetry, transitivity: [PROVEN].

1.5 Hierarchy, pushforward, and the seven measures

Definition 1.14 (Hierarchical scale system). [DEFINITION] A filtration of partitions of $\mathcal{W}$ by depth, $P_0 \preceq P_1 \preceq \dots \preceq P_h$, where each depth- k component decomposes into depth- $(k-1)$ components. *Standing caveat recorded in source:* the refinement law currently concludes **True** — the proper is-submodel-of predicate on POWL v2 objects is [CONJECTURAL].

Definition 1.15 (Pushforward mass and vector measure). [DEFINITION] A *pushforward mass* is $\mu : \mathcal{W} \rightarrow \mathbb{R}_{\geq 0}$. The MFW *vector measure* assigns mass along each of the **seven** distinguished measure kinds:

$$\mu(\cdot) = [\mu_B, \mu_T, \mu_C, \mu_L, \mu_{Sl}, \mu_F, \mu_S]$$

Kind	Symbol	Semantic freedom measured
behavioral	μ_B	$ \{b : \tau(b) = w\} $
temporal	μ_T	temporal realization freedom
choice	μ_C	choice-graph branch mass
linearization	μ_L	PDDL-admitted linearization count
slack	μ_{Sl}	temporal elasticity
fluent	μ_F	object-fluent conditioned mass
entropic	μ_S	fiber-entropic contribution $\eta(w) = p(w) \log F_w $

The seventh kind (**.entropic**) was added by audit: fiber entropy and temporal slack are orthogonal, $\text{Slack}(w) \neq H(B \mid W = w)$. *Provenance requirement (recorded, not yet discharged):* each component should be derived from τ and ν , not stored as an arbitrary nonnegative function.

Definition 1.16 (Additive entropic contribution). [DEFINITION] $\log |F_w|$ is not additive over disjoint fibers ($\log(|F_1| + |F_2|) \neq \log |F_1| + \log |F_2|$); the additive atom is

$$\eta(w) = p(w) \cdot \log |F_w|, \quad H(B \mid W) = \sum_w \eta(w), \quad \eta(C) = \sum_{w \in \text{Leaves}(C)} \eta(w).$$

2 Concurrency: Independence, Traces, Serialization Entropy

Source module: **Concurrency.lean**. The organizing thesis: *the PDDL 3.1 $\rightarrow$ POWL v2 transformation is causal constraint erasure* — remove every ordering distinction not forced by planning truth. A total-order plan contains too much order; POWL v2 is the quotient produced by erasing accidental chronology.

2.1 Independence

Definition 2.1 (IndependenceRelation). [DEFINITION] $I \subseteq A \times A$, symmetric. Two actions commute when $\delta_b(\delta_a(s)) = \delta_a(\delta_b(s))$ on the admitted state domain. The PDDL 3.1 *temporal independence witness* requires five components for (a, b) :

EffectsCommute $\wedge$ PreconditionStable $\wedge$ InvariantStable $\wedge$ NumericFlowCompatible $\wedge$ TrajectoryPreserved.

The dependence relation is $D = (A \times A) \setminus I$.

2.2 Mazurkiewicz traces

Definition 2.2 (TraceEquiv). [DEFINITION] For sequences $u, v \in A^*$, trace equivalence $u \sim_I v$ is the reflexive–symmetric–transitive closure of the adjacent swap

$$u \cdot a \cdot b \cdot v \sim_I u \cdot b \cdot a \cdot v \quad \text{whenever } (a, b) \in I.$$

Theorem 2.3 (traceEquiv.equivalence). [PROVEN] $\sim_I$ is an equivalence relation.

Remark 2.4 (The compression claim). With n pairwise-independent actions, sequence semantics sees $n!$ linearizations while the trace quotient sees one concurrency class: $\text{PDDL} \rightarrow \text{POWL}$ is potentially *factorial* state-space compression by quotienting interleavings. The stronger claim recorded in the source — POWL as a candidate normal form for equivalence classes of lawful plan interleavings — is a research position, not a theorem.

2.3 Causal partial orders and serialization entropy

Definition 2.5 (CausalOrder, linear extensions). [DEFINITION] A *causal order* on n events is an irreflexive transitive $\prec \subseteq \text{Fin}(n)^2$. The least causal relation preserving correctness is the erasure target

$$R^* = \arg \min_R |R| \quad \text{subject to} \quad \text{Lin}(E, R) \subseteq \text{Valid}(\Pi),$$

where $\text{Lin}(P)$ denotes linear extensions: bijections σ with $i \prec j \implies \sigma(i) < \sigma(j)$. Write $e(P) = |\text{Lin}(P)|$ (`linearExtensionCount`).

Definition 2.6 (Serialization entropy). [DEFINITION]

$$H_{\text{ser}}(P) = \log e(P).$$

$H_{\text{ser}} = 0$ means the process is totally forced; large H_{ser} means many serializations correspond to one causal object. The transformation’s distinction-erasure gain for one concurrency class is $G(P) = \log e(P)$.

Definition 2.7 (Temporal serialization entropy). [DEFINITION] For a simple temporal network N (difference bounds $\ell_{ij} \leq t_j - t_i \leq u_{ij}$),

$$\text{Lin}_T(P, N) = \{\ell \in \text{Lin}(P) : \ell \text{ admits a schedule satisfying } N\}, \quad H_T(P, N) = \log |\text{Lin}_T(P, N)|,$$

and the *temporal restriction gap* $H_{\text{ser}}(P) - H_T(P, N)$ measures how much apparent causal concurrency is removed by metric temporal restriction. (In Lean, H_T is currently `opaque`: interface only.) This gap is the formal reason time cannot be omitted from the theory.

2.4 Width, concurrency complex, abstraction ladder

Definition 2.8 (Antichain and width). [DEFINITION] $A \subseteq E$ is an antichain iff its elements are pairwise $\prec$ -incomparable; the width is $w(P) = \max\{|A| : A \text{ antichain}\}$ (by Dilworth, the minimum chain cover size). One scalar is too weak; the intended refinement is local antichain structure over workflow regions, leading to concurrency mass distributions and multifractal analysis of concurrency.

Definition 2.9 (Concurrency complex). [DEFINITION] Given jointly-concurrent admissibility $\text{conc} : \mathcal{P}(\text{Fin } n) \rightarrow \text{Prop}$ with downward closure and $\text{conc}(\emptyset)$ supplied as admission hypotheses (the audit removed sorry'd closure proofs — they are false for arbitrary predicates),

$$K_\Pi = \{A \subseteq E : \text{conc}(A)\}$$

is an abstract simplicial complex, with dimension $\dim K = \max_{A \in K} |A| - 1$. *Falsifier recorded in source*: the concurrency-complex thesis fails if K_Π is trivially contractible for all admitted planning theories. Homological claims: [CONJECTURAL].

Definition 2.10 (Temporal abstraction ladder). [DEFINITION] $\text{MetricTime} \rightarrow \text{IntervalTime} \rightarrow \text{CausalTime} \rightarrow \text{HierarchicalTime}$; each level erases different temporal distinctions, and the sufficient abstraction is determined by the question ($\text{TimeComplete}(Q) \neq \text{TimeComplete}(\text{World})$). Interval orders: $a \prec_I b \iff e_a < s_b$ over intervals $I_a = [s_a, e_a]$; the full chain is $\text{TemporalPlan} \rightarrow \text{IntervalOrder} \rightarrow \text{CausalOrder} \rightarrow \text{POWL}$.

Definition 2.11 (Four concurrency types). [DEFINITION] C_C (causal: unordered in $\prec$), C_T (temporal: intervals may overlap), C_R (resource), and *executable* concurrency $C_E = C_C \cap C_T \cap C_R$.

3 The Kernel: Center of MFW

Source module: `Kernel.lean`. Everything downstream becomes derived mathematics once the kernel is established; without it, τ is an arbitrary classifier and every downstream object is a named aspiration.

3.1 The four equivalence layers

Definition 3.1 (K1: State equivalence). [DEFINITION] $\text{StateEquiv}(b_1, b_2) \iff \text{stateTraceOf}(b_1) = \text{stateTraceOf}(b_2)$ (state sequences agree, ignoring temporal realization). Equivalence laws: [PROVEN].

Definition 3.2 (K2: Causal equivalence). [DEFINITION] With $\text{inducedCausalOrder}(b, I)$ the causal order extracted from I (currently *opaque*: $a \prec b$ iff dependent and earlier),

$$\text{CausalEquiv}_I(b_1, b_2) \iff \exists h : |\mathbf{e}_1| = |\mathbf{e}_2|, \forall i j : \prec_{b_1}(i, j) \leftrightarrow \prec_{b_2}(i^h, j^h).$$

Definition 3.3 (K3: Trace equivalence on lawful behaviors). [DEFINITION]

$$\text{PDDL31TraceEquiv}_I(b_1, b_2) \iff \mathbf{e}_1 \sim_I \mathbf{e}_2.$$

Equivalence laws: [PROVEN].

Definition 3.4 (Load-bearing target: swap preserves lawfulness). [CONJECTURAL]

$$\text{TraceSwapPreservesLawful} : \mathbf{e}_1 \sim_I \mathbf{e}_2 \implies (\text{IsLawful}_\Pi(b_1) \implies \text{IsLawful}_\Pi(b_2)).$$

This is the bridge from the algebraic independence relation to the planning-theoretic admission law. Stated as a `Prop`-valued definition, not proved.

Definition 3.5 (K4: The kernel). [DEFINITION]

$$\text{KernelEquiv}_\tau(b_1, b_2) \iff \tau(b_1) = \tau(b_2), \quad \text{written } b_1 \equiv_K b_2.$$

Equivalence laws and $\equiv_K = \sim_\tau$ definitionally: [PROVEN].

3.2 The crown theorem

Conjecture 3.6 (Crown, forward: τ respects trace equivalence). [CONJECTURAL] $\text{PDDL31TraceEquiv}_I(b_1, b_2) = b_1 \equiv_K b_2$ (soundness: accidental serialization does not affect classification).

Conjecture 3.7 (Crown, partial reverse: kernel refines state equivalence). [CONJECTURAL] $b_1 \equiv_K b_2 \implies \text{StateEquiv}(b_1, b_2)$ (τ does not merge state-inequivalent behaviors).

Conjecture 3.8 (Crown biconditional / kernel characterization). [CONJECTURAL]

$$\boxed{\tau(b_1) = \tau(b_2) \iff \text{PDDL31TraceEquiv}_I(b_1, b_2)} \quad \text{i.e.} \quad \ker(\tau) = \text{TraceEq}_I.$$

Theorem 3.9 (Non-circularity witnesses). [PROVEN] *The two sides are independently defined: `kernelEquiv_unfolds_without_traces` shows $\text{KernelEquiv}_\tau(b_1, b_2) = (\tau(b_1) = \tau(b_2))$ with no reference to independence or traces; `traceEquiv_unfolds_without_tau` shows $\text{PDDL31TraceEquiv}_I(b_1, b_2) = (\mathbf{e}_1 \sim_I \mathbf{e}_2)$ with no reference to τ or $\mathcal{W}$. Both are `rfl`. The biconditional therefore connects two independently specified relations; if proved, it is a structural correspondence, not a definitional unfolding.*

3.3 Consequences conditional on the crown

Theorem 3.10 (`observable_iff_kernel_constant`). [PROVEN] (Definitional preview.) P constant on kernel classes $\iff P$ constant on τ -fibers.

Definition 3.11 (Horizon crossing). [DEFINITION] $\text{crossesHorizon}_\tau(P) \iff \exists b_1 b_2 : b_1 \equiv_K b_2 \wedge P(b_1) \wedge \neg P(b_2)$.

Theorem 3.12 (`fiber_eq_traceClass`, conditional). [PROVEN] under hypothesis *Let $[b]_I = \{b' : \text{PDDL31TraceEquiv}_I(b, b')\}$. If the kernel characterization holds (taken as an explicit hypothesis `hKernel`), then*

$$F_{\tau(b)} = [b]_I.$$

The Lean proof is unconditional given the hypothesis; the hypothesis itself is the crown conjecture.

Conjecture 3.13 (Trace class $\cong$ linear extensions). [CONJECTURAL] *Under the bijection hypotheses (events distinct/`Nodup`, decidable I , independence exactly complementing transition-level dependence):*

$$[b]_{\text{TraceEq}_I} \simeq \text{Lin}(P_b),$$

where $P_b = \text{inducedCausalOrder}(b, I)$. Known in concurrency theory (Mazurkiewicz 1977; Diekert–Rozenberg 1995) but not yet formalized under PDDL 3.1 semantics.

Conjecture 3.14 (Entropy collapse). [CONJECTURAL] *If both the crown and the bijection close, then*

$$S_\tau(\tau(b)) = \log |F_{\tau(b)}| = \log |[b]_I| = \log |\text{Lin}(P_b)| = \log e(P_b) = H_{\text{ser}}(P_b) :$$

fiber entropy equals serialization entropy, collapsing two independently derived quantities and potentially identifying D_q^{Entropic} with $D_q^{\text{Linearization}}$ under some admitted profile. In Lean the statement shell (`FiberEntropyEqSerializationEntropy`) currently concludes `True`: it is a stated target whose conclusion is not yet encoded.

3.4 Kernel generators

Definition 3.15 (KernelGenerator, KernelPath). [DEFINITION] Primitive distinction-erasing moves:

$$g \in \{ \text{traceSwap}, \text{choiceNormalization}, \text{hierarchyCollapse}, \text{temporalAbstraction} \}.$$

A *kernel path* $b_1 \xrightarrow{g_1} b' \xrightarrow{g_2} \dots \xrightarrow{g_n} b_2$ carries its generator sequence, intermediates, and endpoint proofs; its generator-count vector is $\kappa(\text{path}) = (n_{\text{trace}}, n_{\text{choice}}, n_{\text{hier}}, n_{\text{temp}})$.

Conjecture 3.16 (Kernel generated). [CONJECTURAL] $b_1 \equiv_K b_2 \iff \exists \text{ kernel path } b_1 \rightsquigarrow b_2$, i.e. $\ker(\tau) = \text{EqClosure}(\{g_{\text{trace}}, g_{\text{choice}}, g_{\text{hierarchy}}, g_{\text{temporal}}\})$. If it holds, independent generators yield independent spectrum coordinates (one D_q per independent erasure mode) — the proof-first alternative to declaring measure kinds by enumeration.

4 Observability and the Information Horizon

Source module: `Observability.lean` (Layer 5).

Definition 4.1 (Observable / hidden / fiber-constant). [DEFINITION] For $P : \mathcal{P}(\Pi) \rightarrow \text{Prop}$:

$$\begin{aligned} \text{IsObservable}_\tau(P) &\iff \exists \hat{P} : \mathcal{W} \rightarrow \text{Prop}, \forall b : P(b) \leftrightarrow \hat{P}(\tau(b)); \\ \text{IsFiberConstant}_\tau(P) &\iff \forall b_1 b_2 : \tau(b_1) = \tau(b_2) \implies (P(b_1) \leftrightarrow P(b_2)); \\ \text{IsHidden}_\tau(P) &\iff \exists b_1 b_2 : \tau(b_1) = \tau(b_2) \wedge P(b_1) \wedge \neg P(b_2). \end{aligned}$$

Theorem 4.2 (observable_iff_fiber_constant). $\text{IsObservable} \iff \text{IsFiberConstant}$. Forward direction [PROVEN]; reverse direction implemented via the factoring predicate $\hat{P}(w) := \forall b, \tau(b) = w \rightarrow P(b)$ — the source labels this direction [CONJECTURAL] on account of fiber-representative choice, though the stated proof term closes in Lean.

Theorem 4.3 (hidden_iff_not_fiber_constant; observable_not_hidden). $\text{Hidden} \iff \text{not fiber-constant}$ (reverse direction labeled [CONJECTURAL] in-source, classical witnesses); $\text{observable} \implies \text{not hidden}$ [PROVEN].

Definition 4.4 (Observable sigma-algebra). [DEFINITION] $S \subseteq \mathcal{P}(\Pi)$ is τ -saturated iff membership is fiber-constant. The observable sigma-algebra is

$$\mathcal{O}_\tau = \{ S \subseteq \mathcal{P}(\Pi) : S \text{ saturated} \} = \{ \tau^{-1}(U) : U \subseteq \mathcal{W} \},$$

the largest sigma-algebra making τ measurable. Both inclusions (pullbacks are saturated; saturated sets are pullbacks): [PROVEN].

Theorem 4.5 (Boolean closure). [PROVEN] Observables are closed under $\neg, \wedge, \vee, \rightarrow$, under arbitrary binary connectives $f(P, Q)$ and unary $g(P)$, and contain $\top, \perp$ and every $\hat{P} \circ \tau$.

Definition 4.6 (Sufficiency and semantic residual). [DEFINITION] τ is *sufficient* for Y iff Y is observable (deterministic Fisher–Neyman fragment). The stochastic form is

$$I(Y; B \mid W) = H(Y \mid W) - H(Y \mid B) = H(Y \mid W) = 0,$$

formalized only as a placeholder structure (`SemanticResidualInfo`, `residual` ≥ 0); the bridge “zero residual $\implies$ sufficiency” is [CONJECTURAL] (needs measure-theoretic conditional entropy).

Definition 4.7 (Information horizon). [DEFINITION] H_τ = the observable/hidden dichotomy, packaged with $\text{isWithin}(P) \iff \text{IsObservable}_\tau(P)$ (canonical construction [PROVEN]).

Definition 4.8 (Refinement order). [DEFINITION] $\tau_1 \sqsubseteq \tau_2$ (“ τ_1 refines τ_2 ”) iff $\tau_1(b_1) = \tau_1(b_2) \implies \tau_2(b_1) = \tau_2(b_2)$, equivalently $\tau_2 = \psi \circ \tau_1$. Monotonicity (refined transformations observe more), reflexivity, transitivity: [PROVEN].

5 Fiber Entropy (Layer 6)

Source module: `FiberEntropy.lean`. Finite/uniform model with behaviors a finite enumeration of $\mathcal{P}(\Pi)$.

Definition 5.1 (Cardinality, entropy, probability). [DEFINITION]

$$|F_w| = |\{b \in \text{behaviors} : \tau(b) = w\}|, \quad S_\tau(w) = \log |F_w| \text{ (nats)}, \quad p(w) = \frac{|F_w|}{|\mathcal{P}(\Pi)|}.$$

Probabilistic generalization $S_\tau(w) = H(B \mid W = w)$: future work.

Standing of the structural lemmas (as recorded in-source; several carry complete Lean proofs but retain a CONJECTURAL docstring label pending API review):

Statement	Standing
$ F_w > 0$ when w reachable	[PROVEN]
$S_\tau(w) \geq 0$	[CONJECTURAL] (proof term present)
$S_\tau(w) = 0 \iff F_w \leq 1$; for reachable w : $\iff F_w = 1$	[CONJECTURAL] (proof term present)
$ F_{w_1} \leq F_{w_2} \implies S_\tau(w_1) \leq S_\tau(w_2)$	[CONJECTURAL] (proof term present)
S_τ congruence under $w_1 = w_2$	[PROVEN]
total collapse: all $b \mapsto w_{\text{top}}$ gives $S = \log \mathcal{P}(\Pi) $	[CONJECTURAL] (proof term present)
$S_\tau(w) \leq \log \mathcal{P}(\Pi) $	[CONJECTURAL] (proof term present)

Definition 5.2 (Total fiber entropy). [DEFINITION]

$$H(B \mid W) = \sum_w p(w) S_\tau(w), \quad 0 \leq H(B \mid W) \leq \log |\mathcal{P}(\Pi)|,$$

lower bound at injective τ , upper at total collapse. Nonnegativity: proof present, labeled [CONJECTURAL] via its dependency.

Definition 5.3 (Entropy measure and hierarchy). [DEFINITION] $\mu_S(w) = S_\tau(w)$ defines a `PushforwardMass`; $\mu_S(C) = H(B_C \mid W_C)$ distributes entropy over hierarchical components, feeding $D_q^{\text{FiberEntropy}}$. The injection `withFiberEntropySlack` replaces the slack component of a vector measure with S_τ , witnessing the entropic reading $\mu_{Sl}(w) = S_\tau(w)$.

Definition 5.4 (Scaling models and the boundary sufficiency hypothesis). [DEFINITION] With $|C|$ the component size (proxy: $|\partial C| + 1$) and $|\partial C|$ the boundary-variable count:

$$\text{volume: } S_\tau(C) \sim \alpha |C|, \quad \text{area: } S_\tau(C) \sim \beta |\partial C|, \quad \text{mixed: } S_\tau(C) \sim \alpha |C|^\gamma |\partial C|^\delta.$$

Boundary sufficiency (the Bekenstein–Hawking analogue for process models):

$$\exists \beta > 0 : \forall C, \quad |S_\tau(C) - \beta |\partial C|| \leq \varepsilon.$$

Whether real PDDL 3.1 domains exhibit area or volume scaling is an open empirical/theoretical question.

6 Intrinsic Dimension Loss (Layer 7)

Source module: `IntrinsicDimension.lean`.

Definition 6.1 (Coordinates and stratification). [DEFINITION] Phase-space coordinates come in five families:

$$x = (\text{event occurrence, temporal, object-fluent, numeric, trajectory}).$$

Admissibility imposes $C(x) = 0$ and $G(x) \leq 0$; the lawful space is *stratified*,

$$\mathcal{P}(\Pi) = \bigcup_{\sigma \in \Sigma} P_\sigma,$$

Σ indexing discrete choice patterns; each stratum carries a local dimension $\dim P_\sigma$ (continuous freedom: temporal slack, numeric state). Covering law required; overlaps only on boundaries.

Definition 6.2 (Local dimensions and dimension loss). [DEFINITION] $d_{\text{loc}}^{\text{PDDL}}(b) = \max\{\dim P_\sigma : b \in P_\sigma\}$; $d_{\text{loc}}^{\text{POWL}}(w)$ — currently the placeholder 0 (untimed structural POWL). The *transformation dimension loss*:

$$\Delta d_\tau(b) = d_{\text{loc}}^{\text{PDDL}}(b) - d_{\text{loc}}^{\text{POWL}}(\tau(b)) \in \mathbb{Z}.$$

Dimension loss is stronger than cardinality: an uncountable fiber may be low-dimensional; a finite fiber of k points has dimension 0.

Standing: $\Delta d_\tau \geq 0$ — stated with a proof that currently succeeds only because the POWL side is the placeholder 0; the honest statement needs a Lipschitz hypothesis, [CONJECTURAL]. “ $\Delta d_\tau(b) = 0 \iff \tau$ locally invertible at b ” and “fiber dimension = dimension loss under submersion” are [STRUCTURALLY BLOCKED]: they require smooth structure on $\mathcal{P}(\Pi)$ and $\mathcal{W}$ that is not formalized; both are recorded as `Prop` shells with placeholder right-hand sides.

Definition 6.3 (Null perturbation space). [DEFINITION]

$$\text{Null}_\tau(b) = \{\delta : \tau(b \oplus \delta) = \tau(b)\}, \quad \text{Null}_\tau(b) = \bigoplus_k \text{Null}_\tau^k(b), \quad k \in \{\text{temp, fluent, num, traj}\},$$

the τ -invisible lawful perturbations; at smooth points this is the fiber tangent space, $\dim \text{Null}_\tau(b) = \dim F_{\tau(b)} = \Delta d_\tau(b)$. Proven basics: the trivial perturbation is null; each kind-component is a subset; null perturbations preserve $\sim_\tau$ and stay in the fiber. The direct-sum decomposition $\Delta d_\tau = \Delta d^{\text{temp}} + \Delta d^{\text{fluent}} + \Delta d^{\text{num}} + \Delta d^{\text{traj}}$ is [STRUCTURALLY BLOCKED] (needs tangent-space decomposition); the per-kind dimension function is a placeholder returning 0.

Definition 6.4 (Profile and effective scaling). [DEFINITION] $\Delta D_\tau(w) = \sup_{b \in F_w} \Delta d_\tau(b)$ (proxied by fiber dimension); effective scaling dimension $\alpha(w) = d_{\text{loc}}^{\text{PDDL}}(b) - \dim F_{\tau(b)}$ with $\alpha \leq d_{\text{loc}}^{\text{PDDL}}$ [PROVEN] (by $\mathbb{N}$ -subtraction). Measure connection (informal, in-source): $\mu(B_\varepsilon(w)) \sim \varepsilon^{d-k} \text{Vol}_k(F_w \cap B_\varepsilon)$ for source dimension d and fiber dimension k .

7 Observable Basis (Layer 8)

Source module: `ObservableBasis.lean`. Inspired by Energy Flow Polynomials (EFPs), which give a complete linear basis for infrared/collinear-safe observables in collider physics.

Definition 7.1 (Invariant observables). [DEFINITION]

$$F_\tau = \{f : \mathcal{P}(\Pi) \rightarrow \mathbb{R} \mid \forall b_1 b_2, \tau(b_1) = \tau(b_2) \implies f(b_1) = f(b_2)\}.$$

Equivalently $f = g \circ \tau$; the analogue of infrared safety. Bundled form: `InvariantObservable` $= (f, \text{invariance proof})$, with extensionality [PROVEN].

Theorem 7.2 (Algebra of F_τ). [PROVEN] F_τ contains 0 and all constants and is closed under $+$, scalar $\cdot$, negation, and pointwise product; the three subspace axioms are packaged (`invariant_observable_subspa` whose docstring defers Mathlib `Submodule` plumbing). Pullbacks: every $g \circ \tau \in F_\tau$ [PROVEN]; conversely every $f \in F_\tau$ factors through surjective τ via choice of fiber representatives (`invariant_observable_factor` — proof term present, labeled [CONJECTURAL] in-source). Hence, at the intended level of generality, $F_\tau \cong (\mathcal{W} \rightarrow \mathbb{R})$; when $\mathcal{W}$ is finite, $\dim F_\tau = |\text{im } \tau|$.

Definition 7.3 (Primitive kinds and basis). [DEFINITION] Candidate generators by kind (with their EFP analogies):

Kind	Measures	EFP analogy
causal	causal partial-order incidence	angular structure
interval	partial-order interval lengths	energy flow
motif	choice-graph subgraph motif counts	graph polynomials
hierarchical	depth/width/composition	jet substructure
temporal	boundary separation, slack	thrust-like variables

The span of $\vec{\varphi} = [\varphi_1, \dots, \varphi_n]$: $\text{span}\{\varphi_i\} = \{f : \exists \vec{c}, f(b) = \sum_i c_i \varphi_i(b)\}$; $\text{span} \subseteq F_\tau$ has a complete induction proof (labeled [CONJECTURAL] in-source). A *basis* for the admitted space F_τ^{adm} requires membership, spanning ($F_\tau \subseteq \text{span}$), and linear independence (`IsObservableBasis`). The conjecture: the five kinds with products and linear combinations generate F_τ^{adm} . [CONJECTURAL]

Definition 7.4 (Admission profiles and truncation). [DEFINITION] An *admission profile* (`maxDepth`, `maxBranching`) all positive, bounds the theory. An *observable truncation* records surviving kinds, per-kind counts, and a truncated basis, with bound fields (conjectured, carried as structure constraints):

$$\#\text{causal} \leq \binom{\text{maxConcurrency}}{2}, \quad \#\text{motif} \leq 2^{\binom{\text{maxBranching}}{2}},$$

$$\text{truncationOrder} \leq \text{maxConcurrency}^2 + 2^{\text{maxBranching}} + \text{maxDepth} \cdot \text{maxBranching} + \text{maxDuration}.$$

This is proof-directed observable truncation: the analogue of EFP power counting. [CONJECTURAL]

Definition 7.5 (Feature vectors). [DEFINITION] `evaluateBasis`($\vec{\varphi}, b$) $= [\varphi_1(b), \dots, \varphi_n(b)]$. Fiber-mates produce equal feature vectors and the length equals the basis size: both [PROVEN]. Kind correspondence to measures: behavioral/entropic $\rightarrow$ causal, temporal/slack $\rightarrow$ temporal, choice $\rightarrow$ motif, linearization $\rightarrow$ interval, fluent $\rightarrow$ hierarchical. Measure–observable duality ($\mu_k(w) = \Lambda_k(\delta_w)$ for linear functionals Λ_k on F_τ): stated with a trivial witness ($\Lambda = 0$), [CONJECTURAL] in substance.

8 Spectrum Bundle and Cross-Spectrum Tension (Layer 9)

Source module: `SpectrumBundle.lean`.

Definition 8.1 (Multifractal workflow spectrum bundle). [DEFINITION] For each Rényi exponent $q \in \mathbb{R}$ and measure kind i , a generalized dimension $D_q^i \in \mathbb{R}$:

$$\vec{D}_q = (D_q^B, D_q^T, D_q^C, D_q^L, D_q^{Sl}, D_q^F),$$

a section of a trivial $\mathbb{R}^6$ -bundle over the q -line, with the structural axiom that D_0^i is finite for every kind. *Provenance requirement (audit)*: the bundle must be manufactured from admitted measure + scale + fit evidence, not constructed as an arbitrary function of q .

Definition 8.2 (Cross-spectrum tension). [DEFINITION]

$$\Delta_{ij}(q) = D_q^i - D_q^j.$$

Antisymmetry $\Delta_{ij} = -\Delta_{ji}$, diagonal vanishing $\Delta_{ii} = 0$, zero trace: all [PROVEN]. Interpretation table (classification thresholds $\pm\varepsilon$, `classifyTension`):

Pattern	Diagnosis
$D_q^B \gg D_q^T$	temporal bottleneck
$D_q^B \ll D_q^T$	scheduling elasticity
$D_q^C \gg D_q^B$	choice sparsity (dead branches)
$D_q^L \gg D_q^T$	false concurrency pressure

The tension profile, not any single spectrum, is the intended optimization signal.

Definition 8.3 (Hierarchical partition sum). [DEFINITION] Over the POWL v2 filtration $P_0 \preceq \dots \preceq P_h$ with component masses p_B :

$$Z(q, k) = \sum_{B \in P_k} p_B^q.$$

$Z(0, k) = |P_k|$ (proof term present, requires $p > 0$; labeled [CONJECTURAL]); $Z(1, k) = 1$ under normalization [PROVEN]; nonnegativity and additivity over concatenation: proof terms present, labeled [CONJECTURAL]. Log-convexity in q : open question.

Definition 8.4 (The q -lens). [DEFINITION] q is attention control over behavioral mass inside POWL v2 choice structure: $q \gg 0$ spotlights dominant channels; $q = 0$ is democratic counting; $q = 1$ is the Shannon lens; $q \ll 0$ spotlights rare behaviors. `QLensWorkflow` = (bundle, focus kind, q) evaluates to D_q^{kind} ; `qLensChoice` is the branching-structure lens.

Definition 8.5 (q -lens deformation; `QLens.lean` §30). For a positive finite distribution p on finite α :

$$L_q(p)(x) = \frac{p(x)^q}{\sum_y p(y)^q}.$$

Well-definedness (positivity, normalization): [PROVEN]. Ratio law [PROVEN] (`qLens_ratio_law`):

$$\frac{L_q(p)(x)}{L_q(p)(y)} = \left(\frac{p(x)}{p(y)} \right)^q.$$

Definition 8.6 (Workflow metric and information profile). [DEFINITION] A `WorkflowMetric` is $(d, d \geq 0, d(w, w) = 0, \text{symm}, \text{triangle})$, inducing a `PseudoMetricSpace` (construction labeled [CONJECTURAL] pending the distance’s own well-definedness). Note this module’s docstring still describes the superseded Jaccard reading $d_{\mathcal{W}} = 1 - J$; the operative construction is Wasserstein (§9). The *Transformation Information Profile* is the master diagnostic

$$I_{\tau} = (d_{\text{int}}, \vec{D}_q, H(B \mid W), I(Y; B \mid W), \dim F_w, d_{\mathcal{W}}, \Delta_{ij}(q)),$$

with the cached tension provably consistent with the bundle [PROVEN]. The scale-resolved bundle $\vec{D}_q(k)$ adds hierarchical depth; antisymmetry persists at every depth [PROVEN].

9 Workflow Geometry via Conditional Measures

Source: `Kernel.lean` (geometry section).

Definition 9.1 (Behavior metric and Wasserstein workflow distance). [DEFINITION] Given a metric $d_{\mathcal{P}}$ on $\mathcal{P}(\Pi)$ (nonnegative, symmetric, triangle), define conditional fiber measures $\nu_w = \nu(\cdot \mid \tau = w)$ and

$$d_{\mathcal{W}}(w_1, w_2) = W_p(\nu_{w_1}, \nu_{w_2}) = \inf_{\pi \in \Pi(\nu_{w_1}, \nu_{w_2})} \mathbb{E}_{\pi}[d_{\mathcal{P}}(b_1, b_2)].$$

Fibers are disjoint as sets (`fiber_disjoint`), but their behavior distributions may be geometrically close in $d_{\mathcal{P}}$ — that is the optimal-transport repair of the Jaccard collapse. In Lean the distance is `opaque` (interface; the transport construction needs measure theory beyond the module’s scope).

10 Specification and Governance Modules

These modules (Notation Authority §§1–40) formalize the surrounding manufacturing discipline. They are intentionally interface-level: most carriers are `opaque` types, and the properties are `Prop`-valued specifications, not proven theorems.

10.1 Manufacture (§§1–6, 15)

Opaque carriers: observation space $\mathcal{O}$, admitted observations $\mathcal{O}^{\text{adm}}$, artifact domain $\mathcal{A}$; relations: manufacturing law $\mathcal{L} \subseteq \mathcal{A} \times \mathcal{O}$, standing stand $\subseteq \mathcal{A}$, receipts receipt $\subseteq \mathcal{A} \times \mathcal{O}^{\text{adm}}$, actuations with `isReceipted`. The *BRCE invariant* (Zero Unreceipted Actuation):

$$\text{brce} \iff \forall a \in \text{Actuation} : \text{isReceipted}(a).$$

Quality: defect vectors `DefectVector` = `Option DefectVectorImpl`, voice-of-customer `Voice`, CTQ derivation `ctq : Voice → DefectVector`; Rice containment `rice` $\subseteq \mathcal{A}$ (semantic-domain constraint acknowledging Rice’s theorem: verify artifacts within an admitted domain rather than deciding arbitrary semantic properties). All [DEFINITION].

10.2 Falsification (§38)

Status inductive $\{\text{Admitted}, \text{Revoked}\}$; opaque empirical falsifier $F \subseteq \text{Transformation} \times \text{DefectEvidence}$, status lookup, and revocation $\text{revoke} : \text{Context} \times \text{Transformation} \rightarrow \text{Context}$. For context c , transformation T , evidence E :

$$\begin{aligned} \text{RevCond}(c, T, E) &\iff \text{status}(c, T) = \text{Admitted} \wedge F(T, E), \\ \text{FalsLoop}(c, T, E) &\iff (\text{RevCond}(c, T, E) \implies \text{status}(\text{revoke}(c, T), T) = \text{Revoked}). \end{aligned}$$

Both [DEFINITION] (specifications of the admission governance loop); the test module proves the composition $\text{Admitted} \wedge F \wedge \text{FalsLoop} \implies \text{Revoked}$ from these hypotheses.

10.3 Explore/Exploit (§§7–12, 35)

Opaque spaces Ω (open realization space), observations, contracts; noncomputable opaque maps $\text{explore} : \text{Obs} \rightarrow \text{Contracts}$, $\text{exploit} : \text{Contracts} \rightarrow \Omega_C$ where $\Omega_C = (\Omega \rightarrow \text{Prop})$ is a realization class; equivalent-closure selection $\Omega_C \rightarrow \text{EquivClosure}$. *Strict separation*:

$$\forall o_1 o_2 : \text{explore}(o_1) = \text{explore}(o_2) \implies \text{exploit}(\text{explore}(o_1)) = \text{exploit}(\text{explore}(o_2)),$$

i.e. exploitation factors through contracts, never touching raw observations. [DEFINITION] (trivially satisfiable as stated — it is a congruence — but it pins the *interface* shape).

10.4 Search portfolio (QLens.lean §§31–35)

Budget apportionment $\sum_i \text{alloc}(i) = B$ (conservation carried as a proof field). Search rails: an exact completeness carrier F_0 plus exploitation rails $\{q_j\} \subset \mathbb{R}$ (q-lens-parameterized). With uninterpreted Served , $\text{DestructivelyInterferes}$, Complete :

$$\text{PersistentService}(F_0) \iff \forall k \exists t > k : \text{Served}(F_0, t),$$

$$\text{Noninterference}(P) \iff \forall q \in P : \neg \text{DI}(\text{exploit}_q, F_0).$$

Conjecture 10.1 (Portfolio completeness). [CONJECTURAL] $\text{Complete}(F_0) \wedge \text{PersistentService}(F_0) \wedge \text{Noninterference}(P) \implies \text{PortfolioComplete}(P)$. *Requires an operational semantics for rails.*

10.5 Compiler pipeline (§§36–37)

Opaque targets TTLGraph , TeraTemplate , RustExecutable ; opaque functors

$$\text{projectTtl} : \text{Coords} \rightarrow \text{TTL}, \quad \text{interpolateTera} : \text{TTL} \rightarrow \text{Tera}, \quad \text{compileRust} : \text{Tera} \rightarrow \text{Exe}.$$

Complexity coordinates $(\text{time}, \text{space}, \text{multiplicativeDepth}) \in \mathbb{N}^3$. The *Multiplicative Cascade Wind Tunnel* is a record carrying the three stage-equalities as proof fields ($g = \text{projectTtl}(x)$, $t = \text{interpolateTera}(g)$, $e = \text{compileRust}(t)$), and the complexity bound

$$\text{windTunnelComplexityBound} \iff \text{multiplicativeDepth} \leq \text{time}.$$

[DEFINITION]. Test module instantiates a concrete tunnel with **rfl** proofs.

10.6 Central theorem ledger (§§39–40)

Meta-mathematical DAG: nodes (id, statement), edges; ledger = DAG + proved-node subset; a *canonical derivation* carries a sequence with a validity proof. *Known stub*: `validTopologicalSort(dag, seq) := True` — the real acyclicity/order check is not yet written, so ledger tests instantiate the derivation with `True.intro` and validate nothing about ordering. [DEFINITION] (placeholder).

11 Verified Test Surface

`Tests.lean` constructs concrete instances: a two-block blocks-world `PlanningTheory` (full transition function, goal A on B); a four-action theory with exactly one independent pair ($a_1 \perp a_2$) on which $[a_1, a_2] \sim_I [a_2, a_1]$ is [PROVEN] by a single swap, and reflexivity/symmetry/transitivity are exercised on concrete lists; a two-chain causal order on four events with width-2 antichain facts proved; non-circularity #checks; the `fastStart` (hidden: timestamp-dependent) vs. `reachesGoal` (observable) property pair (stated, not yet run through `IsObservable`); generator-count computations by `native.decide`; distinctness and exhaustiveness of all seven `MeasureKinds`. One negative statement — that $[a_1, a_3]$ and $[a_3, a_1]$ are *not* trace-equivalent through a dependent pair — is commented out as [CONJECTURAL] (requires a no-swap-path argument). The unit test files instantiate the governance modules with toys; `ExploreTests` manufactures inhabitants of opaque types via `unsafeCast`, a pattern that bypasses type safety and is flagged for replacement.

12 How the Mathematics Shapes the Implementation

This section maps each formal structure to a concrete constraint or design decision for the `ggen`-based PDDL 3.1 $\rightarrow$ POWL v2 compiler. The compiler chain itself is fixed by §36–37: PDDL 3.1 semantics are lowered to a TTL (Turtle/RDF) semantic graph, `ggen` interpolates Tera templates from the graph, and the templates compile to a Rust executable — with the wind-tunnel record demanding that each stage equality be carried as evidence, not asserted.

12.1 The kernel decides what the compiler may merge

The crown target $\ker(\tau) = \text{TraceEq}_I$ is a two-sided implementation contract:

- **Soundness (forward direction)** — the POWL v2 constructor in `ggen` must be invariant under adjacent swaps of I -independent actions. Concretely: build the workflow object from the *causal partial order* P_b (and choice/hierarchy data), never from the raw event list; any code path that consults raw order beyond $\prec$ breaks the forward direction.
- **Completeness (reverse direction)** — the constructor must *not* merge behaviors that differ causally. Hash-consing or normalization passes in the emitter must be justified by one of the four kernel generators (`traceSwap`, `choiceNormalization`, `hierarchyCollapse`, `temporalAbstraction`); a merge with no generator path is a completeness bug by definition (`KernelGenerated`).

The non-circularity theorems matter operationally: the test oracle for the compiler (trace equivalence, computed from PDDL semantics and I) is independent of the code under test (the emitter), so conformance testing is not self-referential.

12.2 Independence extraction is the load-bearing analysis

Everything routes through I . The five-component witness (effects commute, preconditions stable, invariants stable, numeric flow compatible, trajectory preserved) is the specification of the static analysis **ggen** must run over ground actions. Two consequences:

- Under-approximating I (declaring dependent what is independent) is *safe but lossy*: the emitted partial order over-sequentializes, and the loss is measurable as unrealized serialization entropy $H_{\text{ser}}(P)$.
- Over-approximating I is *unsound*: it licenses swaps that break lawfulness, violating **TraceSwapPreservesLa**. The bijection hypotheses (**independence_exact**) say exactly which notion of dependence must be complemented: transition-level commutation over reachable states.

The temporal restriction gap $H_{\text{ser}}(P) - H_T(P, N)$ dictates that the scheduler cannot be an afterthought: causal concurrency the temporal network forbids (“false concurrency,” also visible as $\Delta_{LT}(q) \gg 0$) must be pruned at emission or the runtime will discover it as deadline misses.

12.3 Fibers are disjoint: geometry and caching consequences

fiber_disjoint is proven, so any implementation feature that assumes overlapping fibers (similarity search by fiber intersection, Jaccard-based workflow clustering) is wrong by theorem. Workflow similarity must be computed as optimal transport between conditional behavior distributions ($W_p(\nu_{w_1}, \nu_{w_2})$) — for the implementation this means retaining per-class behavior samples (or sufficient statistics) rather than only class labels. Conversely, disjointness enables a clean cache: a behavior’s class is a pure function of its trace class, so memoization can key on any canonical representative of $[b]_I$ — e.g. the lexicographically-least linear extension of P_b — and, conditional on the crown, this key is *exactly* right (no false sharing, no misses among equivalent behaviors).

12.4 Entropy accounting as compiler telemetry

The additive atom $\eta(w) = p(w) \log |F_w|$ and the identity $H(B \mid W) = \sum_w \eta(w)$ give the compiler a per-class, hierarchy-summable ledger of erased information. If the entropy collapse closes, $\log |F_w| = H_{\text{ser}}(P_b)$ is computable from the emitted partial order alone (count linear extensions) — no fiber enumeration — turning an information-theoretic diagnostic into a static analysis. The boundary-sufficiency hypothesis is directly testable in the implementation: regress measured $S_\tau(C)$ against $|\partial C|$ (interface variables of each emitted component) versus $|C|$; an area law would justify interface-first component summaries (the POWL boundary suffices to reconstruct interior behavioral information), while a volume law says interior detail must be retained.

12.5 Observability gates what the generated system can report

The Boolean closure theorems plus $\mathcal{O}_\tau = \{\tau^{-1}(U)\}$ mean the emitted Rust artifact can answer, from workflow state alone, exactly the fiber-constant properties — and no engineering on the query layer can recover a hidden one. Practical protocol for **ggen**: every monitoring/analytics requirement Y attached to a domain must be checked for factorization through τ (deterministic sufficiency). **reachesGoal** passes; **fastStart** (timestamp-dependent) fails and requires either carrying timestamps into the POWL object (refining τ , per the **Refines** order — refinement monotonically enlarges the observable algebra, [\[PROVEN\]](#)) or logging outside the workflow rep-

resentation. The refinement partial order is the principled knob: deepen hierarchy or tighten temporal precision until the required Y becomes observable, and no further.

12.6 The observable basis is the feature schema

$F_\tau \cong (\mathcal{W} \rightarrow \mathbb{R})$ says invariant observables are exactly workflow-computable features. The five primitive kinds (causal incidence, interval, motif, hierarchical, temporal-boundary) are the candidate feature generators for process-mining and ML layers over emitted workflows, and **evaluateBasis** (fiber-constant, [PROVEN]) is the specification of the feature extractor: equivalent behaviors must produce identical vectors, which is a directly fuzzable property. The truncation bounds tie feature-set size to the admission profile — $O(\text{maxConcurrency}^2 + 2^{\text{maxBranching}} + \text{depth} \cdot \text{branching} + \text{duration})$ — so bounding the accepted PDDL fragment bounds the feature dimension *a priori* (conjectural constants, but the right shape for capacity planning).

12.7 Spectrum tension as the optimizer’s objective

The formalization is explicit that single spectra are not the optimization signal; the tension profile $\Delta_{ij}(q)$ is. For the implementation this prescribes a diagnostic pass over emitted hierarchies: compute partition sums $Z(q, k)$ from pushforward masses at each depth ($Z(0, k)$ audits component counts, $Z(1, k) = 1$ audits normalization — both good runtime assertions), extract $\vec{D}_q(k)$, and classify tensions: temporal bottleneck $\rightarrow$ scheduler pressure; choice sparsity $\rightarrow$ dead-branch elimination in the choice graph; false concurrency $\rightarrow$ tighten the emitted partial order. The q-lens gives the reviewer dial: $q \gg 0$ audits the hot paths, $q \ll 0$ audits the rare-behavior tail (where lawfulness bugs hide). The q-lens ratio law ([PROVEN]) guarantees the deformation preserves relative mass ordering monotonically in q — reweighting for the portfolio rails cannot invert priorities.

12.8 Governance loop around the compiler

The specification modules dictate the pipeline’s control plane:

- *Explore/exploit separation*: discovery (mining observations into contracts) and generation (contracts into realizations) must communicate only through the contract type — **ggen** may not peek at raw observations during emission.
- *Admission and falsification*: emitted transformations enter a registry as **Admitted**; runtime defect evidence E with $F(T, E)$ triggers **revoke**, and the falsification-loop property is the contract that revocation actually lands (status = Revoked). This is the formal shape of a canary/rollback system for generated code.
- *BRCE*: every actuation of a generated artifact must be receipted — an audit-log invariant the Rust emitter should enforce by construction (no side-effecting entry point without a receipt).
- *Rice containment*: verification claims are scoped to an admitted artifact domain; the implementation should refuse to claim semantic properties outside it.
- *Ledger*: proof/claim dependencies form a DAG with derivations replayed in topological order — but note the current `validTopologicalSort := True` stub: as implemented, the ledger does not yet check ordering, and any tooling built on it must not assume it does.
- *Search portfolio*: generation-time search runs an exact completeness rail alongside q-parameterized exploitation rails, with persistent service and noninterference as the (conjectural) conditions under which adding heuristic rails cannot destroy completeness.

12.9 What is not yet load-bearing

For honest engineering scope: the crown biconditional, swap-preserves- lawfulness, trace-class/linear-extension bijection, entropy collapse, kernel generation, portfolio completeness, boundary sufficiency, and the observable basis completeness are **[CONJECTURAL]**; local-invertibility and dimension-loss decomposition are **[STRUCTURALLY BLOCKED]** on smooth structure; $d_{\text{loc}}^{\text{POWL}}$, per-kind null dimensions, temporal serialization entropy, the Wasserstein distance, and `validTopologicalSort` are placeholders or opaque interfaces. Implementation work may *target* these (they are precise), but no correctness argument may *assume* them yet.

13 Notation Index

Notation	Meaning	Source
Π	planning theory $(S, A, \delta, s_0, G, \Theta, \models_T, N, \models_N, \Gamma, \models_\Gamma)$	Trans
$b = (\mathbf{e}, \mathbf{t})$	behavior trace: events, timestamps	Trans
$\text{IsLawful}_\Pi(b)$	lawfulness predicate (5 clauses)	Trans
$\mathcal{P}(\Pi)$	behavioral phase space (proof-carrying)	Trans
$\text{Powl}(\alpha)$	atom/silent/xor/loop/po model	Trans
$\mathcal{W}$	POWL v2 objects (model, WF, depth, ∂w , choice graphs)	Trans
$\tau : \mathcal{P}(\Pi) \rightarrow \mathcal{W}$	the transformation (bare map)	Trans
$F_w = \tau^{-1}(w)$	fiber	Trans
$F_{w_1} \cap F_{w_2} = \emptyset$	fiber disjointness	Trans
$b_1 \sim_\tau b_2$	transformation equivalence	Trans
$P_0 \preceq \dots \preceq P_h$	hierarchical scale system	Trans
$\mu = [\mu_B, \mu_T, \mu_C, \mu_L, \mu_{SL}, \mu_F, \mu_S]$	7-kind vector measure	Trans
$\eta(w) = p(w) \log F_w $	additive entropic atom	Trans
$I, D = A^2 \setminus I$	independence, dependence	Conc
$u \sim_I v$	Mazurkiewicz trace equivalence	Conc
$P_b, \text{Lin}(P), e(P)$	causal order, linear extensions, count	Conc
$H_{\text{ser}}(P) = \log e(P)$	serialization entropy	Conc
$H_T(P, N)$	temporally feasible entropy	Conc
$w(P)$	causal width (max antichain)	Conc
K_Π	concurrency complex	Conc
$C_E = C_C \cap C_T \cap C_R$	executable concurrency	Conc
$b_1 \equiv_K b_2$	kernel equivalence $\tau(b_1) = \tau(b_2)$	Kernel
$\ker(\tau) = \text{TraceEq}_I$	crown biconditional	Kernel
$[b]_I$	trace class	Kernel
$F_{\tau(b)} = [b]_I$	fiber = trace class	Kernel
$[b]_I \simeq \text{Lin}(P_b)$	trace class/linear extension bijection	Kernel
$g \in \{\text{trace, choice, hierarchy, temporal}\}$	kernel generators	Kernel
$\kappa(\text{path})$	generator count vector	Kernel
$\text{IsObservable, IsHidden, IsFiberConstant}$	property trichotomy	Observer
$\mathcal{O}_\tau$	observable sigma-algebra	Observer
$I(Y; B \mid W)$	semantic residual information	Observer
H_τ	information horizon	Observer

$\tau_1 \sqsubseteq \tau_2$	refinement of transformations	Obser
$S_\tau(w) = \log F_w $	fiber entropy	Fiber
$H(B \mid W) = \sum_w p(w) S_\tau(w)$	total fiber entropy	Fiber
$S_\tau(C) \sim \alpha C $ vs. $\beta \partial C $	volume vs. area scaling	Fiber
$P(\text{Th}) = \bigcup_\sigma P_\sigma$	stratification	Intrin
$\Delta d_\tau(b) = d_{\text{loc}}^{\text{PDDL}} - d_{\text{loc}}^{\text{POWL}}$	dimension loss	Intrin
$\text{Null}_\tau(b), \text{Null}_\tau^k(b)$	null perturbation spaces	Intrin
$\Delta D_\tau(w), \alpha(w)$	loss profile, effective scaling dim.	Intrin
F_τ	invariant observable space	Obser
$f = g \circ \tau$	factorization through τ	Obser
$\text{span}\{\varphi_i\} = F_\tau^{\text{adm}}$	observable basis target	Obser
$\vec{D}_q = (D_q^B, \dots, D_q^F)$	spectrum bundle	Spect
$\Delta_{ij}(q) = D_q^i - D_q^j$	cross-spectrum tension	Spect
$Z(q, k) = \sum_{B \in P_k} p_B^q$	hierarchical partition sum	Spect
$\vec{D}_q(k)$	scale-resolved bundle	Spect
I_τ	transformation information profile	Spect
$L_q(p)(x) = p(x)^q / \sum_y p(y)^q$	q-lens deformation	QLen
$\nu_w = \nu(\cdot \mid \tau = w), W_p(\nu_{w_1}, \nu_{w_2})$	conditional measures, Wasserstein d_W	Kern
brce	zero unreceipted actuation	Manu
RevCond, FalsLoop	revocation condition, falsification loop	Falsif
explore/exploit, Ω_C	strict-separation interface	Explo
F_0 , exploit(q) rails	search portfolio	QLen
projectTtl / interpolateTera / compileRust	compiler cascade	Comp
multDepth $\leq$ time	wind-tunnel complexity bound	Comp
validTopologicalSort	ledger ordering check	Ledge

References

- [1] A. Mazurkiewicz, *Concurrent program schemes and their interpretations*, DAIMI Report PB-78, Aarhus University, 1977.
- [2] V. Diekert and G. Rozenberg (eds.), *The Book of Traces*, World Scientific, 1995.
- [3] H. Kourani and S. J. van Zelst, *POWL: Partially Ordered Workflow Language*, BPM 2023, Definitions 1–2.
- [4] M. Fox and D. Long, *PDDL2.1/3.x: temporal and trajectory extensions of the Planning Domain Definition Language*.
- [5] C. Villani, *Optimal Transport: Old and New*, Springer, 2009.
- [6] P. Komiske, E. Metodiev, J. Thaler, *Energy Flow Polynomials: a complete linear basis for jet substructure*, JHEP 2018.
- [7] The Lean 4 theorem prover and Mathlib; formalization at `procint/ProcInt/MFW`, toolchain `leanprover/lean4:v4.31.0`, Mathlib pinned `fabf563a`.